

Systemy Sztucznej Inteligencji

Dokumentacja projektu – Klasyfikacja przedziału cenowego rosyjskich mieszkań

Kamil Skop, Michał Kosiec, Szymon Sobelski - gr. 8, ?, ?

16 czerwca 2026

Spis treści

1	Część I	2
1.1	Opis programu	2
1.2	Instrukcja obsługi	3
1.3	Opis obsługi	3
2	Część II	4
2.1	Opis działania	4
2.1.1	Przetwarzanie wstępne i binning cen	4
2.1.2	Algorytm KNN	4
2.1.3	Naiwny Bayes	5
2.1.4	Drzewo decyzyjne	5
2.2	Algorytm	6
2.3	Baza danych	7
2.4	Struktura rekordu	7
2.5	Implementacja	8
2.6	Eksperymenty	9
2.7	Wnioski	13

1 Część I

1.1 Opis programu

Celem projektu jest klasyfikacja przedziału cenowego rosyjskich mieszkań na podstawie ich cech (powierzchnia, liczba pokoi, piętro, lokalizacja geograficzna, typ budynku itp.). Dane wejściowe stanowi plik CSV zawierający 200 000 rekordów. Cena każdego mieszkania zostaje zakwalifikowana do jednego z $N = 5$ równo licznych przedziałów cenowych (klas C1–C5) wyznaczanych dynamicznie kwantylami ze zbioru treningowego.

Program losowo, z użyciem stałego `seed = 42` dzieli wczytane dane na zbiór treningowy oraz testowy w proporcji 80:20 po uprzednim oczyszczeniu go z outlierów na podstawie reguły 3-sigma na cechach ciągłych (gdy odległość dowolnej cechy od średniej przekracza 3 odchylenia standardowe, usuwany jest cały rekord).

Program implementuje od zera, bez użycia zewnętrznych bibliotek uczenia maszynowego, trzy klasyfikatory:

- **KNN** (k najbliższych sąsiadów),
- **Naiwny Bayes** (mieszany: Gauss + Bernoulli + wielomianowy),
- **Drzewo decyzyjne** (kryterium Gini, ograniczenie głębokości i minimalnego listka).

W sprawozdaniu przeprowadzono również analizę rezultatów każdego z wytrenowanych klasyfikatorów.

1.2 Instrukcja obsługi

Wymagania:

- Python 3.12+
- Biblioteka `matplotlib`
- Obecność przetworzonej bazy danych z Kaggle

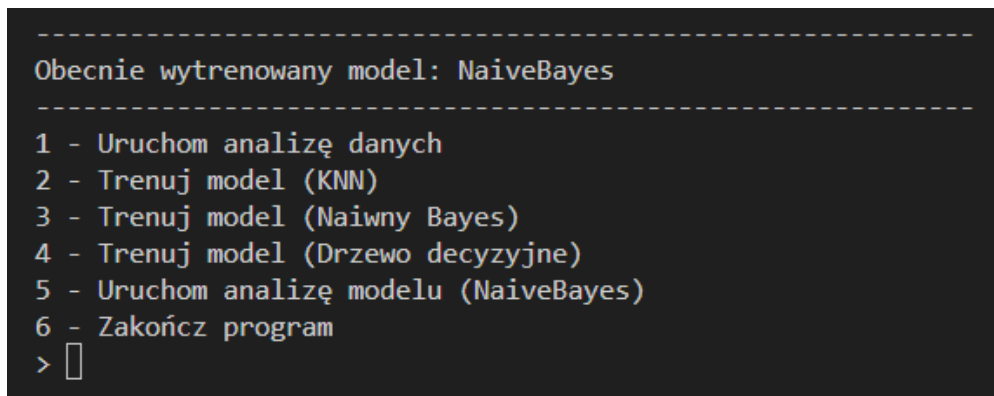
Uruchomienie:

- Upewnić się, że w katalogu projektu znajduje się plik `filtered_v3.csv`. Jeśli ten nie występuje, należy pobrać zbiór danych ze strony Kaggle, a następnie wygenerować go za pośrednictwem skryptu, umieszczonego w katalogu `datalab/`.
- Uruchomić skrypt za pomocą polecenia `python main.py`.

Uwaga: Przetestowanie KNN może wymagać zmniejszenia parametru `EXPECTED_ROWS` w kodzie źródłowym do np. 10 000, ponieważ algorytm ten jest bardzo niewydajny i zwyczajnie nie radzi sobie z większymi ilościami rekordów.

1.3 Opis obsługi

Program wczytuje plik `filtered_v3.csv` z bieżącego katalogu, usuwa outliery i prezentuje menu tekstowe. Należy wybrać interesującą nas opcję, wprowadzając ją z klawiatury i wciskając enter:



```
-----  
Obecnie wytrenowany model: NaiveBayes  
-----  
1 - Uruchom analizę danych  
2 - Trenuj model (KNN)  
3 - Trenuj model (Naiwny Bayes)  
4 - Trenuj model (Drzewo decyzyjne)  
5 - Uruchom analizę modelu (NaiveBayes)  
6 - Zakończ program  
> █
```

Rysunek 1: Zrzut ekranu menu głównego.

1. Analiza danych (wyświetla analizę danych oraz generuje wykresy do katalogów `output_raw/` i `output_clean/`).
2. Trening modelu KNN.
3. Trening modelu Naiwny Bayes.
4. Trening modelu Drzewo decyzyjne.
5. Analiza wyników aktualnie wytrenowanego modelu (macierz pomyłek, dokładność itd.).
6. Wyjście.

2 Część II

2.1 Opis działania

2.1.1 Przetwarzanie wstępne i binning cen

Cena mieszkania jest wartością ciągłą. Aby problem stał się klasyfikacją, ceny dzielone są na N równo licznych przedziałów (kwantylowych):

$$q_i = \text{kwantyl}\left(\frac{i}{N}\right), \quad i = 1, \dots, N - 1.$$

Dla $N = 5$ otrzymujemy 5 klas C1–C5. Podział wyznaczany jest raz z całego zbioru (po usunięciu outlierów), co gwarantuje równo liczne klasy i eliminuje wpływ wartości ekstremalnych na granice.

Usuwanie outlierów – reguła 3-sigma

Dla każdej cechy liczbowej x_j wyznaczana jest średnia μ_j i odchylenie standardowe σ_j . Rekord uznaje się za outlier, jeżeli dla dowolnej cechy j :

$$|x_j - \mu_j| > 3\sigma_j.$$

2.1.2 Algorytm KNN

Klasyfikator k najbliższych sąsiadów ($k = 7$) przydziela klasę na podstawie ważonego głosowania sąsiadów. Dla pary przykładów a, b stosuje się metrykę hybrydową:

$$d(a, b) = \underbrace{\sqrt{\sum_j \left(\frac{a_j - b_j}{s_j}\right)^2}}_{\text{część ciągła}} + d_{\text{region}} + d_{\text{budynek}} + d_{\text{nowe}}$$

gdzie s_j to odchylenie standardowe j -tej cechy ciągłej (normalizacja), a dodatkowe składniki binarne karzą za różny region (+1,0), typ budynku (+0,5) lub różny status nowego budownictwa (+0,4).

Waga sąsiada odwrotnie proporcjonalna do odległości:

$$w_i = \frac{1}{d_i + \varepsilon}, \quad \varepsilon = 10^{-9}.$$

2.1.3 Naiwny Bayes

Klasyczny Naiwny Bayes zakłada, że wszystkie cechy są warunkowo niezależne i mają rozkład normalny. Diagnoza wizualna rozkładów cech wykazała, że założenie o gaussowskości jest niespełnione dla większości zmiennych:

- **Lokalizacja geograficzna:** rozkład wielomodalny (skupiska miejskie).
- **Miesiąc:** rozkład cykliczny.
- **Liczba pokoi / pięter:** wartości dyskretne ze skokami.

W odpowiedzi wdrożono *mieszany Naiwny Bayes* (Mixed Naive Bayes) z trzema niezależnymi torami prawdopodobieństwa:

1. **Gaussowski** – dla $\log(\text{area} + \varepsilon)$ i $\log(\text{kitchen_area} + \varepsilon)$:

$$P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

2. **Bernoulliego** – dla cechy binarnej `new_building` (wygładzanie Laplace'a, $\alpha = 1$):

$$P(x_j | c) = p_{jc}^{x_j} (1 - p_{jc})^{1-x_j}, \quad p_{jc} = \frac{n_{1jc} + 1}{n_c + 2}.$$

3. **Wielomianowy** – dla pozostałych cech dyskretnych i kategorycznych (rok, miesiąc, piętro, liczba pięter, pokoje, region, typ budynku, hasz geo):

$$P(x_j = v | c) = \frac{n_{vjc} + \alpha}{n_c + \alpha \cdot |V_j|}.$$

Spatial hashing: współrzędne geograficzne zaokrąglane są do 0,1 stopnia (≈ 11 km), co zastępuje kosztowne klasteryzowanie i pozwala traktować lokalizacje jako kategorie.

Klasyfikacja: wybierana jest klasa z maksymalnym logarytmem a posteriori:

$$\hat{c} = \arg \max_c \left[\log P(c) + \sum_j \log P(x_j | c) \right].$$

2.1.4 Drzewo decyzyjne

Drzewo budowane jest metodą zachłanną (CART). Kryterium podziału to nieczystość Gini:

$$\text{Gini}(S) = 1 - \sum_c \left(\frac{|S_c|}{|S|} \right)^2.$$

Najlepszy podział minimalizuje ważoną sumę nieczystości potomków:

$$\text{Gain} = \text{Gini}(S) - \frac{|S_L|}{|S|} \text{Gini}(S_L) - \frac{|S_R|}{|S|} \text{Gini}(S_R).$$

Ograniczenia: maksymalna głębokość = 13, minimalna liczba próbek w liściu = 10. Trening odbywa się na losowej próbce 5000 rekordów (seed = 42).

2.2 Algorytm

Ponizej przedstawiono pseudokod glownego potoku przetwarzania danych.

Data: Plik CSV z danymi mieszkam, liczba klas $N = 5$, proporcja treningu $r = 0,8$

Result: Wytrenowany klasyfikator, wyniki na zbiorze testowym

Wczytaj wszystkie rekordy z pliku CSV;

Wyznacz granice klas cenowych kwantylami z cen wszystkich rekordow;

Przypisz kazdy rekord do klasy **C1–C5**;

Oblicz srednia i odch. stand. kazdej cechy numerycznej;

foreach *rekord* **do**

if *dowolna cecha odbiega o* $> 3\sigma$ **then**

 Oznacz jako outlier;

end

end

Usun outliery ze zbioru;

Przetasuj zbior losowo (seed=42);

Podziel na zbior treningowy (80%) i testowy (20%);

Wybierz model (KNN / Naiwny Bayes / Drzewo decyzyjne);

Trenuj wybrany model na zbiorze treningowym;

foreach *przyklad ze zbioru testowego* **do**

 Przewidz klase cenowa;

 Porownaj z etykieta rzeczywista;

end

Wyswietl macierz pomylek i dokladnosc;

Algorithm 1: Glówny potok algorytmu programu

2.3 Baza danych

Dane wejściowe: plik `filtered_v3.csv` (ok. 15 MB, 200 000 rekordów). Zbiór został uzyskany poprzez przetasowanie i sztuczne zmniejszenie zbioru bazowego, pochodzącego z Kaggle, link: <https://www.kaggle.com/datasets/mrdaniilak/russia-real-estate-20182021> za pomocą skryptu z katalogu `datalab/`. Zawiera on około 5,5 miliona rekordów ogłoszeń sprzedaży mieszkań w Rosji w latach 2018 - 2021, redukowanych i tasowanych do 200 000 wpisów. Jako, że w plikach projektu nie załączono bazy, należy wygenerować ją samodzielnie z użyciem wspomnianego wyżej skryptu oraz pliku pobranego z linku.

2.4 Struktura rekordu

Kolumna	Typ	Opis
price	int	Cena mieszkania (RUB)
date	date	Data wystawienia ogłoszenia (YYYY-MM-DD)
time	time	Czas wystawienia ogłoszenia
geo_lat	float	Szerokość geograficzna
geo_lon	float	Długość geograficzna
region	int	ID regionu Rosji
building_type	int	Typ budynku (więcej informacji dostępnych na Kaggle)
level	int	Piętro mieszkania
levels	int	Liczba pięter w budynku
rooms	int	Liczba pokoi (-1 = kawalerka)
area	float	Powierzchnia całkowita (m ²)
kitchen_area	float	Powierzchnia kuchni (m ²)
object_type	int	1 = rynek wtórny, 11 = nowe budownictwo

2.5 Implementacja

Projekt podzielony jest na następujące pliki:

- `russian_flat.py` – klasy danych (`RussianFlatCsv`, `RussianFlatProps`, `RussianFlat`); wczytywanie i etykietowanie rekordów.
- `preprocessor.py` – usuwanie outlierów (reguła 3-sigma) i losowy podział na zbiory.
- `base_model.py` – abstrakcyjna klasa bazowa `BaseModel`.
- `knn.py` – klasyfikator KNN z hybrydową metryką i ważoną głosowaniem.
- `naive_bayes.py` – mieszany Naiwny Bayes (Gauss + Bernoulli + wielomianowy).
- `decision_tree.py` – drzewo decyzyjne CART z kryterium Gini.
- `data_analysis.py` – analiza statystyczna, macierz korelacji Pearsona, wykresy.
- `main.py` – główna petla menu, orkiestracja całego potoku.

Wybrane fragmenty kodu

Przykład: Wyznaczanie skal do normalizacji w KNN.

Listing 1: Obliczanie skal normalizacyjnych (`knn.py`)

```
1 def _compute_continuous_scales(self, training_data):
2     examples = [_ for _ in training_data]
3     columns = list(zip(*(
4         self._extract_features(flat.data)[0]
5         for flat in examples
6     )))
7     scales = []
8     for column in columns:
9         stdev = statistics.stdev(column) if len(column) > 1 else 0.0
10        if stdev <= 0.0:
11            stdev = 1.0
12        scales.append(stdev)
13    return tuple(scales)
```

Przykład: Trening Naiwnego Bayesa – wygładzanie wariancji dla toru gaussowskiego.

Listing 2: Wygładzanie wariancji (`naive_bayes.py`)

```
1 global_variances = []
2 global_columns = list(zip(*all_continuous_features))
3 for col in global_columns:
4     mean = sum(col) / len(col)
5     var = sum((x - mean)**2 for x in col) / (len(col) - 1)
6     global_variances.append(var)
7 epsilons = [max(1e-4 * var, 1e-9) for var in global_variances]
```

2.6 Eksperymenty

Poniżej przedstawiono wyniki przeprowadzonych eksperymentów, obejmujących trening przygotowanych klasyfikatorów oraz ich ocenę na zbiorze testowym.

KNN

Model KNN (k-Nearest Neighbors) został wytrenowany z parametrem $k = 7$ sąsiadów. Kluczowym elementem implementacji jest niestandardowa funkcja odległości, która łączy cztery składowe dopasowane do typów cech.

- **Cechy ciągłe** – Odległość euklidesowa po znormalizowanych wartościach (podział przez odchylenie standardowe liczone na zbiorze treningowym). Dla powierzchni mieszkania oraz powierzchni kuchni zastosowano uprzednio transformację logarytmiczną. Współrzędne geograficzne przeliczane są na kilometry (mnożnik 111 km/° szerokości; dla długości dodatkowo korygowany cosinusem szerokości geograficznej).
- **Cecha binarna (nowe budownictwo)** – Stała kara 0,4 doliczana, gdy wartość różni się między porównywaną parą.
- **Cecha kategoryczna (region)** – Stała kara 1,0 przy niezgodności regionu.
- **Cecha kategoryczna (typ budynku)** – Stała kara 0,5 przy niezgodności typu.

Głosowanie spośród k sąsiadów odbywa się z ważeniem odwrotnością odległości – bliższe przykłady mają proporcjonalnie większy wpływ na predykcję.

Model osiągnął skuteczność 58%.

```
--- Podsumowanie wyników modelu ---
```

Metryka	Wynik
Poprawne predykcje	3296 / 5725
Dokładność (Accuracy)	57.57%

```
--- Macierz korelacji wyników (confusion matrix) ---
```

Rzeczywiste\Przewidziane	C1 <= 1.8M	C2 1.8M - 2.5M	C3 2.5M - 3.5M	C4 3.5M - 5.6M	C5 > 5.6M
C1 <= 1.8M	758	235	117	32	7
C2 1.8M - 2.5M	293	510	304	58	13
C3 2.5M - 3.5M	89	243	592	268	31
C4 3.5M - 5.6M	14	72	279	629	160
C5 > 5.6M	1	11	43	159	807

Rysunek 2: Macierz pomyłek dla KNN.

Naiwny Bayes

Model Naiwny Bayes został wytrenowany w dwóch wariantach – z założeniem rozkładu Gaussa oraz z mieszaną funkcją prawdopodobieństwa, dobraną na podstawie analizy danych. Zastosowano 3 typy rozkładów prawdopodobieństwa w zależności od kolumn.

- **Rozkład ciągły** – Klasyczny rozkład Gaussa. Czasem niezbędne okazało się wykorzystanie funkcji logarytmu, aby zbliżyć taki rozkład do rozkładu normalnego. Obliczana gęstość prawdopodobieństwa dla krzywej dzwonowej.
- **Rozkład multinomial (dyskretny)** – Dane dyskretne. Prawdopodobieństwo obliczane w sposób klasyczny.
- **Rozkład Bernouliego (prawda / fałsz)** – Szczególny przypadek rozkładu dyskretnego. Prawdopodobieństwo obliczane w sposób klasyczny.

Model osiągnął skuteczności kolejno 39.61% oraz 57.16%. Przy drugiej próbie zaszła poprawa wynosząca Z punktów procentowych, co dowodzi wyższości tej metody nad pierwszą.

```
--- Podsumowanie wyników modelu ---
```

Metryka	Wynik
Poprawne predykcje	15136 / 38208
Skuteczność (Accuracy)	39.61%

```
--- Macierz korelacji wyników (confusion matrix) ---
```

Rzeczywiste\Przewidziane	C1 <= 1.8M	C2 1.8M - 2.5M	C3 2.5M - 3.5M	C4 3.5M - 5.5M	C5 > 5.5M
C1 <= 1.8M	4709	2070	375	441	92
C2 1.8M - 2.5M	2540	3040	686	1397	340
C3 2.5M - 3.5M	1422	2677	1162	1617	988
C4 3.5M - 5.5M	521	1674	1230	1608	2562
C5 > 5.5M	225	755	634	826	4617

Rysunek 3: Macierz pomyłek dla Naive Bayes (założenie rozkładu Gaussa).

```
--- Podsumowanie wyników modelu ---
```

Metryka	Wynik
Poprawne predykcje	21840 / 38208
Dokładność (Accuracy)	57.16%

```
--- Macierz korelacji wyników (confusion matrix) ---
```

Rzeczywiste\Przewidziane	C1 <= 1.8M	C2 1.8M - 2.5M	C3 2.5M - 3.5M	C4 3.5M - 5.5M	C5 > 5.5M
C1 <= 1.8M	5370	1707	468	93	16
C2 1.8M - 2.5M	1739	3980	1922	548	23
C3 2.5M - 3.5M	397	1954	3625	1805	134
C4 3.5M - 5.5M	112	591	1892	3757	1080
C5 > 5.5M	15	55	221	1596	5108

Rysunek 4: Macierz pomyłek dla Naive Bayes (założenie rozkładu mieszanego).

Drzewo decyzyjne

Model Drzewa Decyzyjnego został zbudowany z maksymalną głębokością 13 poziomów oraz minimalną liczbą próbek w liściu równą 10. Ze względu na rozmiar zbioru treningowego podczas uczenia losowo próbkuję się co najwyżej 5 000 przykładów (stałe ziarno losowości = 42), co ogranicza czas budowy drzewa.

Jako kryterium podziału zastosowano indeks Giniego. Węzły obsługują dwa rodzaje podziałów, dobierane automatycznie na podstawie typu cechy.

- **Podział progowy (cechy ciągłe)** – Sprawdzany warunek $val \leq threshold$; próg wyznaczany jest jako punkt środkowy między dwiema sąsiednimi wartościami sortowanymi rosnąco. Dla powierzchni mieszkania oraz powierzchni kuchni zastosowano transformację logarytmiczną.
- **Podział równościowy (cechy kategoryczne)** – Sprawdzany warunek $val == cat_val$; iteracja po wszystkich unikalnych wartościach danej cechy (region, typ budynku).

Model osiągnął skuteczność 54%.

```
--- Podsumowanie wyników modelu ---
+-----+
| Metryka          | Wynik          |
+-----+-----+
| Poprawne predykcje | 20563 / 38208 |
| Dokładność (Accuracy) | 53.82%        |
+-----+-----+

--- Macierz korelacji wyników (confusion matrix) ---
+-----+
| Rzeczywiste\Przewidziane | C1 <= 1.8M | C2 1.8M - 2.5M | C3 2.5M - 3.5M | C4 3.5M - 5.5M | C5 > 5.5M |
+-----+-----+-----+-----+-----+-----+
| C1 <= 1.8M                | 5124        | 1586          | 703            | 205            | 36          |
| C2 1.8M - 2.5M            | 1959        | 3245          | 2236           | 718            | 54          |
| C3 2.5M - 3.5M            | 550         | 1776          | 3626           | 1755           | 208         |
| C4 3.5M - 5.5M            | 136         | 571           | 2218           | 3334           | 1173        |
| C5 > 5.5M                  | 24          | 89            | 313            | 1335           | 5234        |
+-----+-----+-----+-----+-----+-----+
```

Rysunek 5: Macierz pomyłek dla drzewa decyzyjnego.

Porównanie

K-Najbliższych Sąsiadów (KNN) wyróżnia się niestandardową funkcją odległości łączącą heterogeniczne typy cech w jedną miarę – odległość euklidesową dla wartości ciągłych (znormalizowaną odchyleniem standardowym) uzupełniono stałymi karami dla cech binarnych i kategorycznych. Predykcja opiera się na ważonym głosowaniu, w którym bliżsi sąsiedzi mają proporcjonalnie większy wpływ na wynik. Wadą modelu jest konieczność przechowywania całego zbioru treningowego w pamięci oraz liniowy czas predykcji względem jego rozmiaru. Dobór wag kar (0,4 / 0,5 / 1,0) ma charakter heurystyczny i nie wynika z analizy danych.

Naiwny Bayes osiągnął najwyższą skuteczność dzięki trafnemu dopasowaniu rozkładów prawdopodobieństwa do charakteru każdej cechy. Kluczową decyzją projektową było potraktowanie cech pozornie ciągłych (rok, miesiąc, piętro, liczba pięter, liczba pokoi, region) jako kategorycznych z rozkładem Multinomial – co odpowiada rzeczywistości, gdyż przyjmują one skończoną liczbę wartości. Jedynie powierzchnia i powierzchnia kuchni modelowane są rozkładem Gaussa po transformacji logarytmicznej. Współrzędne geograficzne dyskretyzowane są do siatki 11×11 km, co pozwala grupować przestrzennie bliskie ogłoszenia bez obliczania rzeczywistych odległości. Całość wspomagana jest wygładzaniem Laplace’a (eliminacja problemu zerowych prawdopodobieństw) oraz wygładzaniem wariancji globalnej dla cech ciągłych. Obliczenia prowadzone w przestrzeni logarytmicznej zapobiegają niedopełnieniu arytmetycznemu.

Drzewo Decyzyjne jako jedyny model potrafi natywnie obsłużyć cechy kategoryczne poprzez podział równościowy, bez konieczności kodowania numerycznego. Kryterium podziału oparte na indeksie Giniego pozwala na automatyczny dobór najistotniejszych cech na każdym poziomie drzewa. Ograniczeniem jest jednak konieczność losowego próbkowania zbioru treningowego do 5 000 przykładów, co powoduje utratę informacji. Drzewo buduje twarde granice decyzyjne, które słabo uogólniają się na dane z dużą liczbą unikalnych wartości kategorycznych (np. identyfikator regionu z setkami wartości).

2.7 Wnioski

Na podstawie wygenerowanych histogramów z nakładaną krzywą Gaussa stwierdzono:

- Cechy `area` i `kitchen_area` – po logarytmowaniu przybliżają rozkład normalny; traktowane gaussowsko.
- Cechy `geo_lat`, `geo_lon` – wyraźny rozkład wielomodalny (skupiska miast); traktowane jako kategorie przez `spatial hashing`.
- Cechy `publish_month` – rozkład cykliczny; traktowane jako kategorie.
- Cechy `level`, `levels`, `rooms` – wartości dyskretne; traktowane wielomianowo.

Na podstawie wyników eksperymentów stwierdzono:

- W praktyce, najlepszym z wytrenowanych klasyfikatorów okazał się Naiwny Bayes ze względu na swoją wysoką wydajność oraz skuteczność, a także brak problemów z treningiem, biorącym pod uwagę cały zbiór danych wejściowych.
- KNN ze względu na swoją bardzo niską wydajność, pomimo osiągnięcia najwyższej dokładności, w praktycznych zastosowaniach byłby bezużyteczny.

Pełen kod aplikacji

main.py

```
1     import os
2     from utils import Utils
3     from russian_flat import RussianFlat
4     from preprocessor import Preprocessor
5     from data_analysis import DataAnalysis
6     from base_model import BaseModel
7     from dummy_model import DummyModel
8     from knn import Knn
9     from naive_bayes import NaiveBayes
10    from decision_tree import DecisionTree
11
12    BORDER = 60 * "-"
13
14    DATAFILE = "filtered_v3.csv"
15    EXPECTED_ROWS = 200000
16
17    OUT_DIR_RAW = "output_raw/"
18    OUT_DIR_CLEAN = "output_clean/"
19
20    NUM_BINS = 5
21
22    # currently trained model
23    current_model: BaseModel = DummyModel()
24
25    # collected data
26    raw_flats: list[RussianFlat]
27    clean_flats: list[RussianFlat]
28    training_data: list[RussianFlat]
29    test_data: list[RussianFlat]
30
31    def pause(clear: bool = False):
32        print("Nacisnij Enter, aby kontynuowac...")
33        input()
34        if clear:
35            os.system("cls" if os.name == "nt" else "clear")
36
37    def menu() -> bool:
38        global current_model
39
40        print(BORDER)
41        print("Obecnie wytrenowany model: " + current_model.name())
42        print(BORDER)
43        print("1 - Uruchom analize danych")
44        print("2 - Trenuj model (KNN)")
45        print("3 - Trenuj model (Naiwny Bayes)")
46        print("4 - Trenuj model (Drzewo decyzyjne)")
47        print("5 - Uruchom analize modelu (" + current_model.name() + ")")
48        print("6 - Zakoncz program")
49        choice = input("> ")
50
51    def train_and_report(model):
```

```

52     global current_model
53     current_model = model
54     current_model.train(training_data)
55     print(f"Model {current_model.name()} wytrenowany.")
56
57     if choice == "1":
58         DataAnalysis.analyse_data(raw_flats, OUT_DIR_RAW)
59         DataAnalysis.analyse_data(clean_flats, OUT_DIR_CLEAN)
60     if choice == "2": train_and_report(Knn())
61     if choice == "3": train_and_report(NaiveBayes())
62     if choice == "4": train_and_report(DecisionTree())
63     if choice == "5": DataAnalysis.analyse_results(
64         [current_model.predict(flat.data) for flat in test_data], #
65         predictions
66         [flat.price_range for flat in test_data] # real price ranges
67     )
68
69     abort = choice == "6"
70     if not abort:
71         pause(clear=True)
72
73     return not abort
74
75     def main():
76         global raw_flats, clean_flats, training_data, test_data
77
78         print(BORDER)
79         print("Rozpoczeto wstepne przetwarzanie danych...")
80
81         raw_flats = RussianFlat.from_csv_file(DATAFILE, skip=1, num_bins=
82             NUM_BINS)
83         print(f"Zaladowano mieszkania w liczbie {len(raw_flats)}.")
84
85         if len(raw_flats) < EXPECTED_ROWS:
86             print(f"UWAGA: Oczekiwano {EXPECTED_ROWS} wierszy, ale zaladowano {
87                 len(raw_flats)}.")
88
89         if len(raw_flats) > EXPECTED_ROWS:
90             raw_flats = raw_flats[:EXPECTED_ROWS]
91             print(f"UWAGA: Zaladowano wiecej wierszy niz oczekiwano. Uzyto tylko
92                 pierwszych {EXPECTED_ROWS} wierszy.")
93
94         if len(raw_flats) == 0:
95             print("Nie mozna kontynuowac bez danych. Zamykam program.")
96             return
97
98         outlier_indexes = Preprocessor.outlier_indexes([flat.data for flat
99             in raw_flats])
100         delete_count = len(outlier_indexes)
101         delete_ratio = (delete_count / len(raw_flats)) * 100
102         print(f"Zidentyfikowano {len(outlier_indexes)} outlierow ({
103             delete_ratio:.2f}%).")
104
105         clean_flats = Utils.list_without_indexes(raw_flats, outlier_indexes)
106         print(f"Usunieto outliery. Pozostalo {len(clean_flats)} mieszkam.")

```

```

101
102     training_data, test_data = Preprocessor.divide_data_randomly(
103         clean_flats, ratio=0.8)
104     print(f"Podzielono na zbiory treningowy ({len(training_data)}) oraz
105           testowy ({len(test_data)}).")
106
107     pause(clear=True)
108
109     while menu():
110         pass
111
112     if __name__ == "__main__":
113         main()

```

base_model.py

```

1     from abc import ABC, abstractmethod
2     from russian_flat import RussianFlat, RussianFlatProps
3
4     class BaseModel(ABC):
5     def name(self) -> str:
6     return self.__class__.__name__
7
8     @abstractmethod
9     def train(self, training_data: list[RussianFlat]):
10    pass
11
12    @abstractmethod
13    def predict(self, entry: RussianFlatProps) -> str:
14    pass

```

data_analysis.py

```

1     import os
2     import shutil
3     import statistics
4     import matplotlib
5     import math
6     matplotlib.use("Agg")
7     import matplotlib.pyplot as plt
8     from russian_flat import RussianFlat
9     from table_builder import TableBuilder
10    from utils import Utils
11
12    class DataAnalysis:
13
14    @staticmethod
15    def analyse_data(data: list[RussianFlat], output_dir: str):
16    if not data:
17    print("Brak danych do analizy.")
18    return

```

```

19
20 # 1. Zarzadzanie katalogiem wyjsciowym
21 if os.path.exists(output_dir):
22     shutil.rmtree(output_dir)
23     os.makedirs(output_dir)
24
25 # 2. Analiza rozkladu klas cenowych
26 print(f"\n--- Analiza danych ({output_dir}) - liczba rekordow: {len(
27     data)} ---")
28 distribution = {}
29 for flat in data:
30     price_range = flat.price_range
31     distribution[price_range] = distribution.get(price_range, 0) + 1
32
33 builder_dist = TableBuilder(["Przedzial cenowy", "Liczba mieszkancow",
34     "Procentowo"])
35 total = len(data)
36 for price_range, count in sorted(distribution.items()):
37     percentage = (count / total) * 100
38     builder_dist.add_row([price_range, count, f"{percentage:.1f}%"])
39     print(builder_dist.build())
40
41 # POBRANIE CECH CIAGLYCH
42 continuous_names = [
43     field for field in Utils.get_attrs_of(data[0].data, (int, float))
44     if type(getattr(data[0].data, field)) is not bool
45 ]
46
47 # 3. Analiza statystyczna cech ciaglych
48 print("\n--- Statystyki cech ciaglych ---")
49 builder_stats = TableBuilder(["Cecha", "Min", "Max", "Srednia", "
50     Odch. Stand."])
51 for field in continuous_names:
52     values = [getattr(entry.data, field) for entry in data]
53     min_v = round(min(values), 4)
54     max_v = round(max(values), 4)
55     avg_v = round(statistics.mean(values), 4)
56     stdev_v = round(statistics.stdev(values), 4) if len(values) > 1 else
57         0.0
58     builder_stats.add_row([field, min_v, max_v, avg_v, stdev_v])
59     print(builder_stats.build())
60
61 # 4. MACIERZ KORELACJI (Pearson)
62 print("\n--- Macierz Korelacji (Cechy ciagle) ---")
63
64 def color_by_corr(value: float, text: str) -> str:
65     value = max(-1.0, min(1.0, value))
66     intensity = abs(value)
67     base = int(255 * (1 - intensity))
68     if value >= 0:
69         r, g, b = 255, base, base
70     else:
71         r, g, b = base, base, 255
72     brightness = 0.299 * r + 0.587 * g + 0.114 * b
73     fg = 30 if brightness > 128 else 97

```

```

70     return f"\x1b[{fg}m\x1b[48;2;{r};{g};{b}m{text}\x1b[0m"
71
72     builder_corr = TableBuilder(["Cecha" + continuous_names)
73
74     for f1 in continuous_names:
75         row = [f1]
76         vals1 = [getattr(entry.data, f1) for entry in data]
77
78         for f2 in continuous_names:
79             vals2 = [getattr(entry.data, f2) for entry in data]
80
81             if f1 == f2:
82                 row.append(color_by_corr(1.0, "1.00"))
83             else:
84                 try:
85                     corr = statistics.correlation(vals1, vals2)
86                     row.append(color_by_corr(corr, f"{corr:.2f}"))
87                 except statistics.StatisticsError:
88                     row.append("N/A")
89
90     builder_corr.add_row(row)
91
92     print(builder_corr.build())
93
94     # 5. Wykresy
95     print("\n--- Generowanie wykresow ---")
96     try:
97         # Wykres rozkladu klas cenowych
98         fig, ax = plt.subplots(figsize=(8, 5))
99         ranges = [price_range for price_range, _ in sorted(distribution.
100             items())]
101         counts = [distribution[price_range] for price_range in ranges]
102         ax.bar(ranges, counts, color="#4c72b0")
103         ax.set_xlabel("Przedzial cenowy")
104         ax.set_ylabel("Liczba mieszkancow")
105         ax.set_title("Rozklad liczby mieszkancow wedlug przedzialow cenowych")
106         ax.set_xticks(range(len(ranges)))
107         ax.set_xticklabels(ranges, rotation=45, ha="right")
108         fig.tight_layout()
109         fig.savefig(os.path.join(output_dir, "class_distribution.png"), dpi
110             =150)
111         plt.close(fig)
112
113         # Wykres macierzy korelacji
114         fig, ax = plt.subplots(figsize=(max(6, len(continuous_names) * 0.8),
115             max(6, len(continuous_names) * 0.8)))
116         corr_matrix = []
117         for f1 in continuous_names:
118             row = []
119             vals1 = [getattr(entry.data, f1) for entry in data]
120             for f2 in continuous_names:
121                 vals2 = [getattr(entry.data, f2) for entry in data]
122                 if f1 == f2:
123                     row.append(1.0)
124                 else:

```

```

122     try:
123         row.append(statistics.correlation(vals1, vals2))
124     except statistics.StatisticsError:
125         row.append(0.0)
126     corr_matrix.append(row)
127
128     im = ax.imshow(corr_matrix, cmap="bwr", vmin=-1, vmax=1)
129     ax.set_xticks(range(len(continuous_names)))
130     ax.set_yticks(range(len(continuous_names)))
131     ax.set_xticklabels(continuous_names, rotation=45, ha="right")
132     ax.set_yticklabels(continuous_names)
133     ax.set_title("Macierz korelacji cech ciaglych")
134     fig.colorbar(im, ax=ax, label="Korelacja Pearsona")
135
136     for i in range(len(continuous_names)):
137         for j in range(len(continuous_names)):
138             corr_value = corr_matrix[i][j]
139             percent_text = f"{corr_value * 100:.0f}%"
140             ax.text(j, i, percent_text,
141                 ha="center", va="center",
142                 color="black" if abs(corr_value) < 0.5 else "white",
143                 fontsize=8)
144
145     fig.tight_layout()
146     fig.savefig(os.path.join(output_dir, "correlation_matrix.png"), dpi
147                 =150)
148     plt.close(fig)
149
150     # 6. Wykresy rozkladu poszczegolnych cech
151     print("Generowanie wykresow rozkladu poszczegolnych cech...")
152     for field in continuous_names:
153         values = [getattr(entry.data, field) for entry in data]
154         if not values:
155             continue
156
157         fig, ax = plt.subplots(figsize=(8, 5))
158
159         unique_values = set(values)
160         is_discrete = len(unique_values) <= 25 or all(isinstance(v, int) or
161             (isinstance(v, float) and v.is_integer()) for v in values)
162
163         if is_discrete:
164             counts = {}
165             for v in values:
166                 counts[v] = counts.get(v, 0) + 1
167
168             x_vals = sorted(counts.keys())
169
170             total_count = len(values)
171             y_vals = [counts[x] / total_count for x in x_vals]
172
173             ax.bar(x_vals, y_vals, color="#2a9d8f", edgecolor="black", alpha
174                 =0.7, label="Wartosci (rzeczywistosc)")
175
176         if len(x_vals) <= 15:

```

```

174     ax.set_xticks(x_vals)
175     else:
176         step = len(x_vals) // 15
177         ax.set_xticks(x_vals[::step])
178
179     ax.set_title(f"Rozklad wartosci dyskretnej: {field}")
180     else:
181         ax.hist(values, bins=30, density=True, color="#e9c46a", edgecolor="
            black", alpha=0.6, label="Histogram (rzeczywistosc)")
182         ax.set_title(f"Rozklad wartosci ciaglej: {field}")
183
184     if len(values) > 1:
185         mean = statistics.mean(values)
186         stdev = statistics.stdev(values)
187
188     if stdev > 0:
189         x_min, x_max = min(values), max(values)
190         step = (x_max - x_min) / 100 if x_max > x_min else 1
191         x_axis = [x_min + i * step for i in range(101)]
192         y_axis = []
193         for x in x_axis:
194             exponent = math.exp(-0.5 * ((x - mean) / stdev) ** 2)
195             pdf = (1 / (stdev * math.sqrt(2 * math.pi))) * exponent
196             y_axis.append(pdf)
197
198         ax.plot(x_axis, y_axis, color="#e76f51", linewidth=2, label="Krzywa
            Gaussa (zalozenie modelu)")
199         ax.legend()
200
201         ax.set_ylabel("Czestosc / Gestosc prawdopodobienstwa")
202         ax.set_xlabel(field)
203         fig.tight_layout()
204         safe_name = field.replace(" ", "_").replace("/", "_")
205         fig.savefig(os.path.join(output_dir, f"dist_{safe_name}.png"), dpi
            =150)
206         plt.close(fig)
207
208         print(f"Wykresy zapisane do katalogu: \"{output_dir}\".")
209         except Exception as e:
210             print("Blad przy generowaniu wykresow:", e)
211
212         print("\nAnaliza danych zakonczona.")
213
214     @staticmethod
215     def analyse_results(predicted: list[str], real: list[str]):
216         if not predicted or not real or len(predicted) != len(real):
217             print("Blad: Dane przewidywane i rzeczywiste nie pasuja do siebie.")
218             return
219
220         print("\n--- Podsumowanie wynikow modelu ---")
221         correct = sum(1 for p, r in zip(predicted, real) if p == r)
222         total = len(predicted)
223         accuracy = (correct / total) * 100
224
225         builder = TableBuilder(["Metryka", "Wynik"])

```

```

226     builder.add_row(["Poprawne predykcje", f"{correct} / {total}"])
227     builder.add_row(["Dokladnosc (Accuracy)", f"{accuracy:.2f}%"])
228     print(builder.build())
229
230     print("\n--- Macierz korelacji wyników (confusion matrix) ---")
231     labels = sorted(set(real) | set(predicted))
232     confusion = {real_label: {pred_label: 0 for pred_label in labels}
233                  for real_label in labels}
234     for p, r in zip(predicted, real):
235         confusion[r][p] += 1
236
237     max_count = max(
238         confusion[r][p]
239         for r in labels
240         for p in labels
241     ) if labels else 0
242
243     def color_by_value(count: int, text: str) -> str:
244         intensity = count / max_count if max_count else 0.0
245         base = int(255 * (1 - intensity))
246         r, g, b = 255, base, base
247         brightness = 0.299 * r + 0.587 * g + 0.114 * b
248         fg = 30 if brightness > 128 else 97
249         return f"\x1b[{{fg}}m\x1b[48;2;{{r}};{{g}};{{b}}m{{text}}\x1b[0m"
250
251     builder_conf = TableBuilder(["Rzeczywiste\\Przewidziane"] + labels)
252     for real_label in labels:
253         row = [real_label]
254         for pred_label in labels:
255             count = confusion[real_label][pred_label]
256             row.append(color_by_value(count, str(count)))
257         builder_conf.add_row(row)
258     print(builder_conf.build())
259
260     print("Analiza wyników modelu zakończona.")

```

decision_tree.py

```

1     import math
2     import random
3     from collections import Counter
4     from russian_flat import RussianFlat, RussianFlatProps
5     from base_model import BaseModel
6
7     MAX_TRAIN_SAMPLES = 5_000
8     MAX_DEPTH = 13
9     MIN_SAMPLES_LEAF = 10
10
11     class _Node:
12         __slots__ = ('feat', 'threshold', 'cat_val', 'is_cat', 'left', 'right', 'label')
13
14     def __init__(self):

```

```

15     self.feats = None
16     self.threshold = None
17     self.cat_val = None
18     self.is_cat = False
19     self.left = None
20     self.right = None
21     self.label = None
22
23     class DecisionTree(BaseModel):
24     def __init__(self, max_depth: int = MAX_DEPTH, min_samples: int =
        MIN_SAMPLES_LEAF):
25     self.max_depth = max_depth
26     self.min_samples = min_samples
27     self.root: _Node | None = None
28
29     def name(self) -> str:
30     return "Drzewo decyzyjne"
31
32     def train(self, training_data: list[RussianFlat]):
33     print(f"Trenuje drzewo na {min(len(training_data), MAX_TRAIN_SAMPLES
        )} probkach...")
34     if len(training_data) > MAX_TRAIN_SAMPLES:
35     data = random.Random(42).sample(training_data, MAX_TRAIN_SAMPLES)
36     else:
37     data = list(training_data)
38
39     X = [_extract(flat.data) for flat in data]
40     y = [flat.price_range for flat in data]
41     self.root = self._build(X, y, 0)
42
43     def predict(self, entry: RussianFlatProps) -> str:
44     if self.root is None:
45     return "unknown"
46     return _traverse(self.root, _extract(entry))
47
48     def _build(self, X: list, y: list, depth: int) -> _Node:
49     node = _Node()
50     counts = Counter(y)
51     node.label = counts.most_common(1)[0][0]
52
53     if depth >= self.max_depth or len(y) <= self.min_samples or len(
        counts) == 1:
54     return node
55
56     best = self._best_split(X, y, len(y))
57     if best is None:
58     return node
59
60     feat, threshold, cat_val, is_cat = best
61     node.feats = feat
62     node.threshold = threshold
63     node.cat_val = cat_val
64     node.is_cat = is_cat
65
66     if is_cat:

```

```

67     mask = [x[feat] == cat_val for x in X]
68     else:
69     mask = [x[feat] <= threshold for x in X]
70
71     X_l = [x for x, m in zip(X, mask) if m]
72     y_l = [lbl for lbl, m in zip(y, mask) if m]
73     X_r = [x for x, m in zip(X, mask) if not m]
74     y_r = [lbl for lbl, m in zip(y, mask) if not m]
75
76     if not X_l or not X_r:
77     return node
78
79     node.left = self._build(X_l, y_l, depth + 1)
80     node.right = self._build(X_r, y_r, depth + 1)
81     node.label = None
82     return node
83
84     def _best_split(self, X: list, y: list, n: int):
85     base_gini = _gini_counter(Counter(y), n)
86     best_gain = 1e-10
87     best = None
88
89     for feat in range(len(X[0])):
90     vals = [x[feat] for x in X]
91
92     if isinstance(vals[0], str):
93     for cat_val in set(vals):
94     l_labels = [lbl for v, lbl in zip(vals, y) if v == cat_val]
95     r_labels = [lbl for v, lbl in zip(vals, y) if v != cat_val]
96     nl, nr = len(l_labels), len(r_labels)
97     if nl < self.min_samples or nr < self.min_samples:
98     continue
99     gain = base_gini - (nl / n * _gini(l_labels) + nr / n * _gini(
100         r_labels))
101     if gain > best_gain:
102     best_gain = gain
103     best = (feat, None, cat_val, True)
104     else:
105     pairs = sorted(zip(vals, y))
106     right_cnt = Counter(y)
107     left_cnt: Counter = Counter()
108     nl, nr = 0, n
109     prev = None
110
111     for val, lbl in pairs:
112     if prev is not None and val != prev and nl >= self.min_samples and
113         nr >= self.min_samples:
114     gain = base_gini - (
115     nl / n * _gini_counter(left_cnt, nl) +
116     nr / n * _gini_counter(right_cnt, nr)
117     )
118     if gain > best_gain:
119     best_gain = gain
120     best = (feat, (prev + val) / 2.0, None, False)
121     left_cnt[lbl] += 1

```

```

120     right_cnt[lbl] -= 1
121     if right_cnt[lbl] == 0:
122         del right_cnt[lbl]
123     nl += 1
124     nr -= 1
125     prev = val
126
127     return best
128
129
130     def _gini_counter(counter: Counter, n: int) -> float:
131         if n == 0:
132             return 0.0
133         return 1.0 - sum((c / n) ** 2 for c in counter.values())
134
135     def _gini(labels: list) -> float:
136         n = len(labels)
137         if n == 0:
138             return 0.0
139         return 1.0 - sum((c / n) ** 2 for c in Counter(labels).values())
140
141     def _extract(entry: RussianFlatProps) -> list:
142         return [
143             math.log(float(entry.area) + 1e-5),
144             math.log(float(entry.kitchen_area) + 1e-5),
145             float(entry.geo_lat),
146             float(entry.geo_lon),
147             float(entry.level),
148             float(entry.levels),
149             float(entry.rooms),
150             float(entry.publish_year),
151             float(entry.publish_month),
152             1.0 if entry.new_building else 0.0,
153             float(entry.region_id),
154             str(entry.building_type),
155         ]
156
157     def _traverse(node: _Node, features: list) -> str:
158         if node.label is not None:
159             return node.label
160         val = features[node.feats]
161         go_left = (val == node.cat_val) if node.is_cat else (val <= node.
162             threshold)
163         return _traverse(node.left if go_left else node.right, features)

```

dummy_model.py

```

1     from russian_flat import RussianFlat, RussianFlatProps
2     from base_model import BaseModel
3
4     class DummyModel(BaseModel):
5     def train(self, training_data: list[RussianFlat]):
6     pass

```

```

7
8     def predict(self, entry: RussianFlatProps) -> str:
9         return "unknown"

```

knn.py

```

1     import math
2     import statistics
3     from russian_flat import RussianFlat, RussianFlatProps
4     from base_model import BaseModel
5
6     class Knn(BaseModel):
7     def __init__(self, k: int = 7):
8         self.k = k
9         self.training_data: list[RussianFlat] = []
10        self.continuous_scales: tuple[float, ...] = ()
11
12    def train(self, training_data: list[RussianFlat]):
13        self.training_data = training_data[:]
14        self.continuous_scales = self._compute_continuous_scales(self.
            training_data)
15
16    def predict(self, entry: RussianFlatProps) -> str:
17        if not self.training_data:
18            return "unknown"
19
20        entry_features = self._extract_features(entry)
21        distances = []
22
23        for flat in self.training_data:
24            train_features = self._extract_features(flat.data)
25            dist = self._distance(entry_features, train_features)
26            distances.append((dist, flat.price_range))
27
28        distances.sort(key=lambda item: item[0])
29        nearest = distances[: self.k]
30
31        if not nearest:
32            return "unknown"
33
34        return self._vote(nearest)
35
36    def _vote(self, neighbors: list[tuple[float, str]]) -> str:
37        weights: dict[str, float] = {}
38        for dist, label in neighbors:
39            if dist <= 1e-9:
40                return label
41            weights[label] = weights.get(label, 0.0) + 1.0 / (dist + 1e-9)
42
43        return max(weights.items(), key=lambda item: item[1])[0]
44
45    def _distance(
46        self,

```

```

47     first: tuple[tuple[float, ...], bool, str, str],
48     second: tuple[tuple[float, ...], bool, str, str],
49 ) -> float:
50     first_cont, first_new_building, first_region, first_building = first
51     second_cont, second_new_building, second_region, second_building =
        second
52
53     squared_sum = 0.0
54     for value_a, value_b, scale in zip(first_cont, second_cont, self.
        continuous_scales):
55         diff = (value_a - value_b) / scale
56         squared_sum += diff * diff
57
58     numeric_distance = math.sqrt(squared_sum)
59
60     binary_distance = 0.4 if first_new_building != second_new_building
        else 0.0
61     region_distance = 1.0 if first_region != second_region else 0.0
62     building_distance = 0.5 if first_building != second_building else
        0.0
63
64     return numeric_distance + binary_distance + region_distance +
        building_distance
65
66     def _compute_continuous_scales(self, training_data: list[RussianFlat
        ]) -> tuple[float, ...]:
67         if not training_data:
68             return (1.0,) * 9
69
70         examples = [_ for _ in training_data]
71         columns = list(zip(*(self._extract_features(flat.data)[0] for flat
            in examples)))
72
73         scales: list[float] = []
74         for column in columns:
75             stdev = statistics.stdev(column) if len(column) > 1 else 0.0
76             if stdev <= 0.0:
77                 stdev = 1.0
78             scales.append(stdev)
79
80         return tuple(scales)
81
82     @staticmethod
83     def _extract_features(entry: RussianFlatProps) -> tuple[tuple[float,
        ...], bool, str, str]:
84         lat_km = float(entry.geo_lat) * 111.0
85         lon_km = float(entry.geo_lon) * 111.0 * math.cos(math.radians(float(
            entry.geo_lat)))
86
87         continuous = (
88             float(entry.publish_year),
89             float(entry.publish_month),
90             float(entry.level),
91             float(entry.levels),
92             float(entry.rooms),

```

```

93     math.log(float(entry.area) + 1e-5),
94     math.log(float(entry.kitchen_area) + 1e-5),
95     lat_km,
96     lon_km,
97 )
98
99     return continuous, entry.new_building, entry.region_id, entry.
        building_type

```

naive_bayes.py

```

1     import math
2     from collections import defaultdict
3     from russian_flat import RussianFlat, RussianFlatProps
4     from base_model import BaseModel
5
6     class NaiveBayes(BaseModel):
7     def __init__(self):
8     self.class_counts = defaultdict(int)
9     self.class_priors = {}
10
11     self.continuous_stats = defaultdict(list)
12     self.binary_probs = defaultdict(list)
13     self.cat_probs = defaultdict(dict)
14     self.cat_unseen_probs = defaultdict(dict)
15
16     self.num_classes = 0
17
18     def train(self, training_data: list[RussianFlat]):
19     separated_data_cont = defaultdict(list)
20     separated_data_bin = defaultdict(list)
21     separated_data_cat = defaultdict(list)
22
23     all_continuous_features = []
24     global_cat_uniques = defaultdict(set)
25
26     total_samples = len(training_data)
27
28     # 1. Zbieranie i grupowanie danych
29     for flat in training_data:
30     label = flat.price_range
31     cont_features, bin_features, cat_features = self.extract_features(
        flat.data)
32
33     separated_data_cont[label].append(cont_features)
34     separated_data_bin[label].append(bin_features)
35     separated_data_cat[label].append(cat_features)
36     all_continuous_features.append(cont_features)
37     self.class_counts[label] += 1
38
39     for i, val in enumerate(cat_features):
40     global_cat_uniques[i].add(val)
41

```

```

42     self.num_classes = len(self.class_counts)
43
44     # 2. Obliczanie globalnego wygładzania wariancji dla cech ciągłych
45     global_variances = []
46     if total_samples > 1 and all_continuous_features:
47         global_columns = list(zip(*all_continuous_features))
48         for col in global_columns:
49             mean = sum(col) / len(col)
50             var = sum((x - mean) ** 2 for x in col) / (len(col) - 1)
51             global_variances.append(var)
52         else:
53             global_variances = [0.0] * len(all_continuous_features[0])
54
55     epsilons = [max(1e-4 * var, 1e-9) for var in global_variances]
56
57     # 3. Trening parametrów dla każdej klasy
58     for label in self.class_counts:
59         self.class_priors[label] = (self.class_counts[label] + 1) / (
            total_samples + self.num_classes)
60         class_size = self.class_counts[label]
61
62     # 1. Trening: Ciągłe (Gauss)
63     cont_columns = zip(*separated_data_cont[label])
64     cont_stats = []
65     for i, column in enumerate(cont_columns):
66         mean = sum(column) / len(column)
67         if len(column) > 1:
68             variance = sum((x - mean) ** 2 for x in column) / (len(column) - 1)
69         else:
70             variance = 0.0
71         cont_stats.append((mean, variance + epsilons[i]))
72     self.continuous_stats[label] = cont_stats
73
74     # 2. Trening: Binarne (Bernoulli)
75     bin_columns = zip(*separated_data_bin[label])
76     bin_probs = []
77     alpha_bin = 1.0
78     for column in bin_columns:
79         ones_count = sum(column)
80         prob_one = (ones_count + alpha_bin) / (class_size + 2 * alpha_bin)
81         bin_probs.append(prob_one)
82     self.binary_probs[label] = bin_probs
83
84     # 3. Trening: Kategorie (Multinomial)
85     cat_columns = zip(*separated_data_cat[label])
86     self.cat_probs[label] = {}
87     self.cat_unseen_probs[label] = {}
88     alpha_cat = 1.0
89
90     for i, column in enumerate(cat_columns):
91         counts = defaultdict(int)
92         for val in column:
93             counts[val] += 1
94
95     v_total = len(global_cat_uniques[i])

```

```

96     self.cat_probs[label][i] = {}
97
98     for val in global_cat_uniques[i]:
99         prob = (counts[val] + alpha_cat) / (class_size + alpha_cat * v_total
100         )
101     self.cat_probs[label][i][val] = prob
102
103     self.cat_unseen_probs[label][i] = alpha_cat / (class_size +
104         alpha_cat * v_total)
105
106     def predict(self, entry: RussianFlatProps) -> str:
107         cont_features, bin_features, cat_features = self.extract_features(
108             entry)
109
110         best_label = "unknown"
111         max_log_prob = -float('inf')
112
113         for label in self.class_counts:
114             log_prob = math.log(self.class_priors[label])
115
116             # 1. Ciagle (Gauss)
117             stats = self.continuous_stats[label]
118             for x, (mean, var) in zip(cont_features, stats):
119                 log_pdf = -0.5 * math.log(2 * math.pi * var) - ((x - mean) ** 2) /
120                     (2 * var)
121             log_prob += log_pdf
122
123             # 2. Binarne (Bernoulli)
124             probs = self.binary_probs[label]
125             for x, p_one in zip(bin_features, probs):
126                 if x == 1.0:
127                     log_prob += math.log(p_one)
128                 else:
129                     log_prob += math.log(1.0 - p_one)
130
131             # 3. Kategoryczne (Multinomial)
132             for i, val in enumerate(cat_features):
133                 prob = self.cat_probs[label][i].get(val, self.cat_unseen_probs[label]
134                     [i])
135             log_prob += math.log(prob)
136
137             if log_prob > max_log_prob:
138                 max_log_prob = log_prob
139                 best_label = label
140
141         return best_label
142
143     @staticmethod
144     def extract_features(entry: RussianFlatProps) -> tuple[tuple[float,

```

```

145     binary = (
146     1.0 if entry.new_building else 0.0,
147     )
148
149     categorical = (
150     int(entry.publish_year),
151     int(entry.publish_month),
152     int(entry.level),
153     int(entry.levels),
154     int(entry.rooms),
155     int(entry.region_id),
156     str(entry.building_type),
157     f"{round(float(entry.geo_lat), 1)}_{round(float(entry.geo_lon), 1)}"
        , # 11km x 11km
158     )
159
160     return continuous, binary, categorical

```

preprocessor.py

```

1     from utils import Utils
2     from typing import NamedTuple
3     from operator import attrgetter
4     from typing import Callable
5     import statistics
6     import random
7
8     class Preprocessor:
9
10    @staticmethod
11    def outlier_indexes(data: list) -> list[int]:
12    if len(data) <= 1:
13    return []
14
15    continuous_names = Utils.get_attrs_of(data[0], (int, float))
16    getters = {f: attrgetter(f) for f in continuous_names}
17
18    means = {
19        field: statistics.mean(getters[field](entry) for entry in data)
20        for field in continuous_names
21    }
22
23    stdevs = {
24        field: statistics.stdev(getters[field](entry) for entry in data)
25        for field in continuous_names
26    }
27
28    def is_outlier(entry) -> bool:
29    return any(
30    abs(getters[field](entry) - means[field]) > 3 * stdevs[field]
31    for field in continuous_names
32    )
33

```

```

34     return [i for i, entry in enumerate(data) if is_outlier(entry)]
35
36     @staticmethod
37     def divide_data_randomly(data: list, ratio: float) -> tuple[list,
38         list]:
39         data = data[:]
40         rng = random.Random(42) # fixed seed for reproducibility
41         rng.shuffle(data)
42         split_index = int(len(data) * ratio)
43         return data[:split_index], data[split_index:]

```

russian_flat.py

```

1     import statistics
2     import bisect
3     from typing import NamedTuple
4     from datetime import date, datetime, time
5
6     class RussianFlatCsv(NamedTuple):
7         price: int
8         date_value: date
9         time_value: time
10        geo_lat: float
11        geo_lon: float
12        region: int
13        building_type: int
14        level: int
15        levels: int
16        rooms: int
17        area: float
18        kitchen_area: float
19        object_type: int
20
21        @staticmethod
22        def from_csv_line(line: str) -> 'RussianFlatCsv':
23            parts = line.strip().split(",")
24
25            dt = datetime.strptime(
26                f"{parts[1]} {parts[2]}",
27                "%Y-%m-%d %H:%M:%S"
28            )
29
30            return RussianFlatCsv(
31                price=int(parts[0]),
32                date_value=dt.date(),
33                time_value=dt.time(),
34                geo_lat=float(parts[3]),
35                geo_lon=float(parts[4]),
36                region=int(parts[5]),
37                building_type=int(parts[6]),
38                level=int(parts[7]),
39                levels=int(parts[8]),
40                rooms=int(parts[9]),

```

```

41     area=float(parts[10]),
42     kitchen_area=float(parts[11]),
43     object_type=int(parts[12])
44 )
45
46     class RussianFlatProps(NamedTuple):
47     publish_year: int
48     publish_month: int
49     geo_lat: float
50     geo_lon: float
51     region_id: str
52     building_type: str
53     level: int
54     levels: int
55     rooms: int
56     area: float
57     kitchen_area: float
58     new_building: bool
59
60     class RussianFlat(NamedTuple):
61     price_range: str
62     data: RussianFlatProps
63
64     price_dividers: list[float] = []
65
66     @classmethod
67     def set_price_dividers(cls, prices: list[int], num_bins: int):
68     cls.price_dividers = statistics.quantiles(prices, n=num_bins)
69
70     @classmethod
71     def get_dynamic_price_range(cls, price: int) -> str:
72     if not cls.price_dividers:
73     return "unknown"
74
75     divs = cls.price_dividers
76     idx = bisect.bisect_right(divs, price)
77
78     if idx == 0:
79     return f"C{idx+1} <= {divs[0]/1_000_000:.1f}M"
80
81     if idx == len(divs):
82     return f"C{idx+1} > {divs[-1]/1_000_000:.1f}M"
83
84     low, high = divs[idx-1], divs[idx]
85     return f"C{idx+1} {low/1_000_000:.1f}M - {high/1_000_000:.1f}M"
86
87     @classmethod
88     def from_csv_file(cls, filename: str, skip: int, num_bins: int) ->
89     list['RussianFlat']:
89     csv_records = []
90     with open(filename, "r", encoding="utf-8") as f:
91     for i, line in enumerate(f):
92     if i < skip:
93     continue
94     csv_records.append(RussianFlatCsv.from_csv_line(line))

```

```

95
96     if not csv_records:
97         return []
98
99     all_prices = [record.price for record in csv_records]
100     cls.set_price_dividers(all_prices, num_bins)
101
102     flats = []
103     for record in csv_records:
104         flats.append(
105             cls(
106                 price_range=cls.get_dynamic_price_range(record.price),
107                 data=RussianFlatProps(
108                     publish_year=record.date_value.year,
109                     publish_month=record.date_value.month,
110                     geo_lat=record.geo_lat,
111                     geo_lon=record.geo_lon,
112                     region_id=str(record.region),
113                     building_type=cls.building_type_string(record.building_type),
114                     level=record.level,
115                     levels=record.levels,
116                     rooms=record.rooms,
117                     area=record.area,
118                     kitchen_area=record.kitchen_area,
119                     new_building=record.object_type == 11
120                 )
121             )
122         )
123
124     return flats
125
126     @classmethod
127     def from_csv_line(cls, line: str) -> 'RussianFlat':
128         flat_csv = RussianFlatCsv.from_csv_line(line)
129         return cls(
130             price_range=cls.get_dynamic_price_range(flat_csv.price),
131             data=RussianFlatProps(
132                 publish_year=flat_csv.date_value.year,
133                 publish_month=flat_csv.date_value.month,
134                 geo_lat=flat_csv.geo_lat,
135                 geo_lon=flat_csv.geo_lon,
136                 region_id=str(flat_csv.region),
137                 building_type=cls.building_type_string(flat_csv.building_type),
138                 level=flat_csv.level,
139                 levels=flat_csv.levels,
140                 rooms=flat_csv.rooms,
141                 area=flat_csv.area,
142                 kitchen_area=flat_csv.kitchen_area,
143                 new_building=flat_csv.object_type == 11
144             )
145         )
146
147     @staticmethod
148     def building_type_string(building_type: int) -> str:
149         if building_type == 0: return "other"

```

```

150     if building_type == 1: return "panel"
151     if building_type == 2: return "monolithic"
152     if building_type == 3: return "brick"
153     if building_type == 4: return "block"
154     if building_type == 5: return "wood"
155     return "unknown"

```

table_builder.py

```

1     import re
2
3     class TableBuilder:
4         ANSI_ESCAPE = re.compile(r"\x1b\[([0-9;]*)m")
5
6         def __init__(self, header: list):
7             self.clen = len(header)
8             self.column_widths = [0] * self.clen
9             self.rowlists = []
10            self.add_row(header)
11
12            @staticmethod
13            def _strip_ansi(text: str) -> str:
14                return TableBuilder.ANSI_ESCAPE.sub("", text)
15
16            def add_row(self, row: list):
17                if len(row) < self.clen:
18                    row = row + [""] * (self.clen - len(row))
19
20                self.column_widths = [
21                    max(self.column_widths[j], len(self._strip_ansi(str(row[j]))) )
22                    for j in range(self.clen)
23                ]
24                self.rowlists.append(row)
25
26            def build(self) -> str:
27                result = [
28                    self._format_outside_sep(),
29                    self._format_header(),
30                    self._format_inside_sep()
31                ]
32
33                for row in self.rowlists[1:]:
34                    result.append(self._format_row(row))
35
36                result.append(self._format_outside_sep())
37                return "\n".join(result)
38
39            def _format_header(self) -> str:
40                return self._format_row(self.rowlists[0])
41
42            def _format_inside_sep(self) -> str:
43                return self._format_row(["-" * self.column_widths[j] for j in range(
44                    self.clen)])].replace(" ", "-")

```

```

44
45     def _format_outside_sep(self) -> str:
46     return "+" + ("-" * (len(self._format_header()) - 2)) + "+"
47
48     def _format_row(self, row: list) -> str:
49     cells = []
50     for j in range(self.clen):
51     raw = str(row[j])
52     visible = self._strip_ansi(raw)
53     padding = self.column_widths[j] - len(visible)
54     cells.append(raw + " " * padding)
55     return "| " + " | ".join(cells) + " |"

```

utils.py

```

1     class Utils:
2
3     @staticmethod
4     def list_without_indexes(lst: list, indexes: list[int]) -> list:
5     return [item for i, item in enumerate(lst) if i not in indexes]
6
7     @staticmethod
8     def get_attrs_of(record, types: tuple[type, ...]) -> list[str]:
9     all_names = record._fields if hasattr(record, "_fields") else record
10    .__dict__.keys()
11    return [
12    field for field in all_names
13    if isinstance(getattr(record, field), types)
14    ]

```
